# Django

Capítulo 4 Área administrativa

# **Django**
# Capítulo 4 - Área Administrativa

Como vimos anteriormente, o Django nos proporciona uma interface administrativa, onde é possível gerenciarmos os dados que serão armazenados em nosso banco. Inicialmente, o admin pode parecer muito vazio, apenas mostrando os dados através de CRUDs (CREATE, READ, UPDATE e DELETE), mas existem diversas outras coisas que podemos fazer no painel administrativo, como filtros, pesquisas e emissão de relatórios. Veremos como personalizar nosso painel admin ou, como é chamado no Django, nossa model admin para que ela possa ser bem mais do que um painel, no qual vamos inserir e gerenciar dados do nosso sistema. A seguir veremos apenas alguns deles.

# **Django**
# Capítulo 4 - Área Administrativa

Por agora iremos no arquivo loja/**admin.py** e iremos criar uma uma classe que servirá para alterar aparência e comportamento do nosso admin, por isso essa classe que criaremos herdara de outra, chamada ModelAdmin, para implementar essas mudanças. No caso, iremos adicionar um filtro com base na data de criação para nosso model Fabricante. Para isso, criaremos uma nova classe e colocaremos nela uma variável(componente) " date_hierarchy = 'criado_em' " onde o "criado_em" precisa ser um campo DataField. Essa classe deve ser posicionada antes de implementar nossos models no painel do admin, ou seja, antes dos comandos admin.site.register(Model):

```
class FabricanteAdmin(admin.ModelAdmin):
    # Cria um filtro de hierarquia com datas
    date_hierarchy = 'criado_em'
```

# Django
## Capítulo 4 - Área Administrativa

após adicionar essa classe, para de fato implementarmos filtro ser adicionado no site admin, precisamos adicionar essa classe como outro parâmetro na linha que possui o comando de registrar o nosso model de Fabricante na interface do admin("admin.site.register(Fabricante)"): assim:

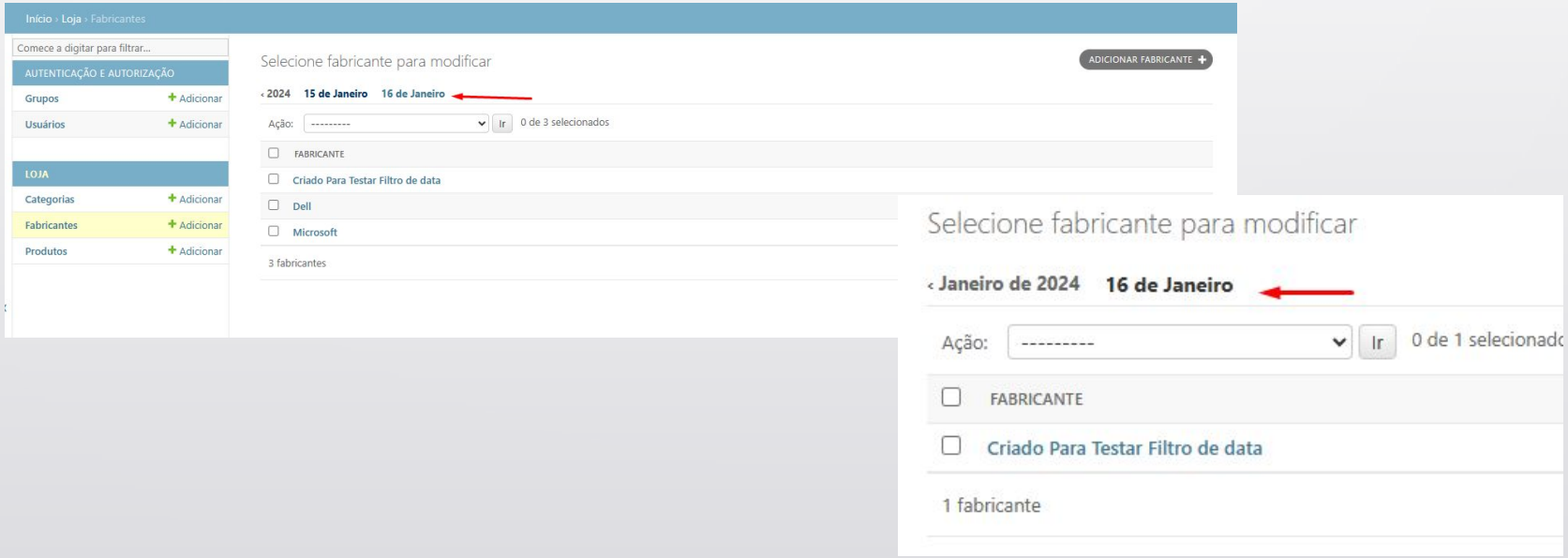
```
admin.site.register(Fabricante,FabricanteAdmin)
```

loja > admin.py > ...

```
1  from django.contrib import admin
2  # Register your models here.
3  from .models import * #imporata nossos models
4  class FabricanteAdmin(admin.ModelAdmin):
5      # Cria um filtro de hierarquia com datas
6      date_hierarchy = 'criado_em'
7  admin.site.register(Fabricante,FabricanteAdmin)
8  admin.site.register(Categoria)
9  admin.site.register(Produto)
```

# Django
## Capítulo 4 - Área Administrativa

Rode a aplicação e verifique  função adcionada.



# Django
## Capítulo 4 - Área Administrativa

Agora adicionaremos uma classe para produto que permitirá ter um filtro por data, mostrar mais informações do registro(objeto ou dado) diretamente na listagem e mudará a apresentação de campos vazios. Código:

```
class ProdutoAdmin(admin.ModelAdmin):
  date_hierarchy = 'criado_em'
  list_display = ('Produto', 'destaque', 'promocao', 'msgPromocao',
'preco', 'categoria',)
  empty_value_display = 'Vazio'
```

Após adicionar a classe, modifique o parâmetro da adição do model de Produto:

```
admin.site.register(Produto, ProdutoAdmin)
```

# Django
## Capítulo 4 - Área Administrativa

Codigo completo:

loja > admin.py > ...

```python
from django.contrib import admin
# Register your models here.
from .models import * #imporata nossos models
class FabricanteAdmin(admin.ModelAdmin):
    date_hierarchy = 'criado_em'
class ProdutoAdmin(admin.ModelAdmin):
    date_hierarchy = 'criado_em'
    list_display = ('Produto', 'destaque', 'promocao', 'msgPromocao', 'preco', 'categoria',)
    empty_value_display = 'Vazio'
admin.site.register(Fabricante,FabricanteAdmin)
admin.site.register(Categoria)
admin.site.register(Produto, ProdutoAdmin)
```

# **Django**
# Capítulo 4 - Área Administrativa

Novos componentes

"**list_display = ('Produto', 'destaque', 'promocao', 'msgPromocao', 'preco', 'categoria',)**" servirá para mostrar esses campos diretamente na listagem

"**empty_value_display = 'Vazio'**" para mostrar o texto "Vazio" sempre que o campo não tiver nada armazenado.

# **Django**
# Capítulo 4 - Área Administrativa

No nosso modelo de produto permitimos que o campo “msgPromocao” tenha um valor nulo na classe modelo Produto.py, porém não possui o parâmetro que permite deixar o campo dele no formulário em branco. Para permitir isso devemos adicionar o parâmetro "blank=True" no campo desejado. Assim:

```
msgPromocao = models.CharField(null=True, max_length=100, blank=True)
```

Depois podemos usar os comandos de preparação e implementação da modificação no banco de dados pelo cmd, com o ambiente virtual ativo:

```
python manage.py makemigrations loja
python manage.py migrate loja
```

# Django
## Capítulo 4 - Área Administrativa

Prompt de Comando - python manage.py runserver 127.0.0.1:8080

```
(venv) C:\Loja>python manage.py makemigrations loja
Migrations for 'loja':
  loja\migrations\0004_alter_produto_msgpromocao.py
    - Alter field msgPromocao on produto

(venv) C:\Loja>python manage.py migrate loja
Operations to perform:
  Apply all migrations: loja
Running migrations:
  Applying loja.0004_alter_produto_msgpromocao... OK

(venv) C:\Loja>python manage.py runserver 127.0.0.1:8080
```

O produto "mouse" foi adicionado com sucesso.

Selecione produto para modificar

ADICIONAR PRODUTO +

‹ 2024 15 de Janeiro 16 de Janeiro

Ação: --------- Ir 0 de 3 selecionados

| PRODUTO | DESTAQUE | PROMOCAO | MSGPROMOCAO | PRECO | CATEGORIA |
|---|---|---|---|---|---|
| mouse | ✗ | ✗ | Vazio | 50,00 | Hardware |
| Windows 11 | ✗ | ✓ | Só hoje! | 1,99 | Software |
| Monitor 24 polegadas | ✗ | ✓ | Até proximo sabado! | 1001,25 | Hardware |

3 produtos

# **Django**
# Capítulo 4 - Área Administrativa

Outros componentes podem ser usados também para modificar o comportamento da classe no admin basta os adicionar na classe. Exemplo de alguns componentes:

O componente fields permite informar que campos deverão ser exibidos no formulário. Os campos do tipo not null deverão ser adicionados na listagem para poder enviar os dados sem erro. `fields = ('Produto', 'destaque', 'promocao', 'msgPromocao', 'preco', 'categoria',)`

O componente search_fields define um campo que servirá para o sistema pesquisar na lista de registros . `search_fields = ('Produto',)`

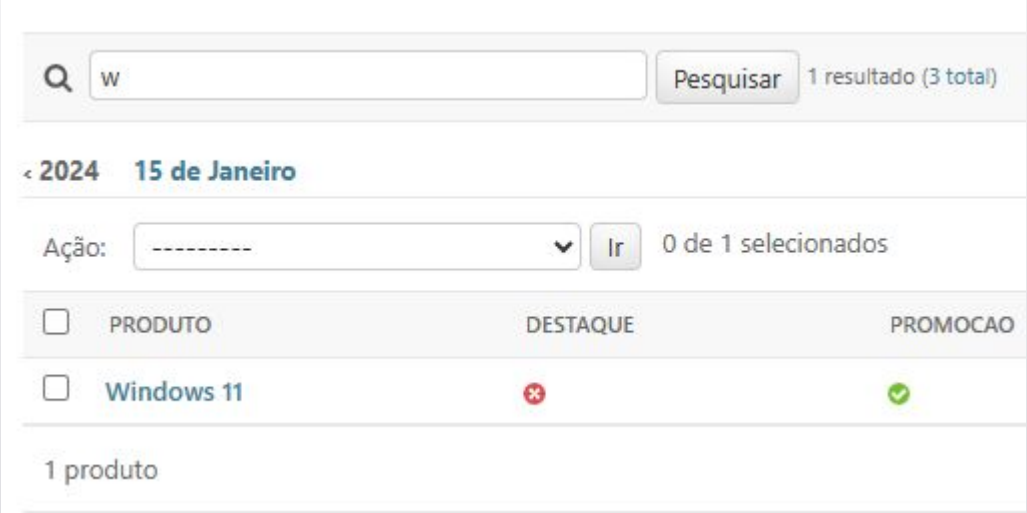
# **Django**
## Capítulo 4 - Área Administrativa

O componente exclude removerá do formulário os campos indicados, e, se o campo pode ser nulo, a validação do blank pode ser ignorado,ou seja mesmo com a campo não tendo nada escrito, a interface não impedirá de criar o objeto. `exclude = ('msgPromocao',).`

OBS: Esses componentes podem ser adicionados dentro das classes que criamos para modificar o painel do adm, mas tem que estar de acordo com os campos do model. O "exclude" e "fields" podem entrar em conflito quando juntos . Além desses, no capítulo 4.1, do livro de referência possui outros componentes interessantes para se usar dependendo do projeto.

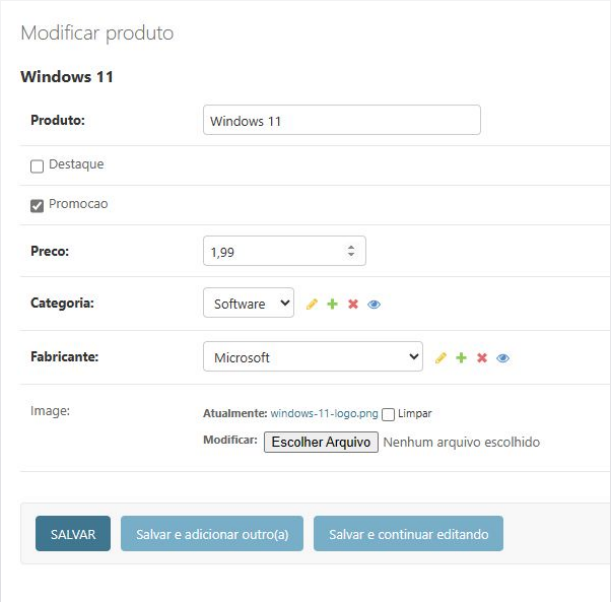
# Django
## Capítulo 4 - Área Administrativa

Usando o `search_fields = ('Produto',)`:



# Django
## Capítulo 3 - Área Administrativa

Usando o `fields = ('Produto', 'destaque', 'promocao', 'msgPromocao', 'preco', 'categoria',):`

# Django
## Capítulo 3 - Área Administrativa

Usando o `exclude = ('msgPromocao',)` :